SIndex: An SSD-based Large-scale Indexing with Deterministic Latency for Cloud Block Storage

Shucheng Wang
China Mobile (Suzhou) Software Technology Co., Ltd.
Suzhou, China
wangshucheng@cmss.chinamobile.com

Kaiye Zhou
China Mobile (Suzhou) Software Technology Co., Ltd.
Suzhou, China
zhoukaiye@cmss.chinamobile.com

Zhandong Guo
China Mobile (Suzhou) Software Technology Co., Ltd.
Suzhou, China
guozhandong@cmss.chinamobile.com

Qiang Cao✉
Huazhong University of Science and Technology
Wuhan, China
caoqiang@hust.edu.cn

Jun Xu
China Mobile (Suzhou) Software Technology Co., Ltd.
Suzhou, China
xujun@cmss.chinamobile.com

Jie Yao
Huazhong University of Science and Technology
Wuhan, China
jackyao@hust.edu.cn

ABSTRACT

The Solid State Drives (SSD) based key-value stores face significant challenges in achieving deterministic access latency. We experimentally observe the long-tail latency is mainly caused by I/O blocking induced by SSD's internal tasks. In this paper, we propose an SSD-based SIndex to store hundreds of billions of block-mapping entries for the latency-critical cloud block storage. To hide the latency fluctuations induced by garbage collection and buffer flushing, SIndex proposes an inter-SSD I/O scheduling based on read/write separation and SSD state transition, while adopting opportunistic request speculation and balancing mechanism to mitigate read disturbance and I/O contention. Moreover, SIndex introduces a write-staging buffer cache and a two-stage sync mechanism to preferentially buffer updated data before synchronizing them to SSDs. We evaluate the SIndex prototype with a variety of benchmarks and real-world traces on commodity SSDs. SIndex is demonstrated to outperform RocksDB and other approaches by up to 11.2× in tail latency without affecting the throughput performance.

ACM Reference Format:
Shucheng Wang, Kaiye Zhou, Zhandong Guo, Qiang Cao, Jun Xu, and Jie Yao. 2024. SIndex: An SSD-based Large-scale Indexing with Deterministic Latency for Cloud Block Storage. In *The 53rd International Conference on Parallel Processing (ICPP '24), August 12–15, 2024, Gotland, Sweden.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3673038.3673041

1 INTRODUCTION

Solid State Drives (SSD) demonstrate a significant enhancement in throughput and latency compared to traditional Hard Disk Drives (HDD). The SSD-based key-value stores such as RocksDB [7] have been widely deployed in production environments, including high-performance computing [29, 32], cloud storage [22] and big data analysis [31], to provide nearly unbound storage capacity that far exceeds the main memory while satisfying the performance and availability requirements of a myriad of application and tenants.

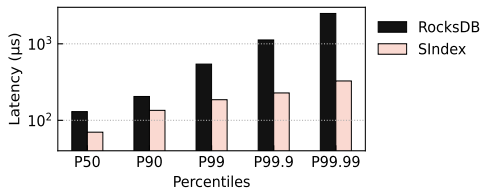


Figure 1: Median and tail Latency of SSD-based indexes

However, managing the tail latency and performance consistency on these systems for meeting Service Level Agreements (SLAs) is a critical problem. Fig.1 shows the latency performance on SSD-based RocksDB under *Put*/*Get* mixed workloads, which is 14.1× higher than their normal state (detailed in Section 3.1). The SSD-based KV stores often overlook the impact of SSD devices on latency predictability. The internal activities within SSDs, such as garbage collection [13, 30], buffer flushing [14, 18] and read disturbance [10] would inevitably trigger massive background I/Os and exhibit serious long latency. This complicates the use of SSD-based indexes for latency-sensitive applications.

To tame the long-tail I/O latency on SSDs, there have been efforts on program suspension [17, 33], GC coordination [30] and evolving device interfaces [21], which usually require significant modifications to the firmware and may be unattractive for quick deployment on commercial SSDs. Recently, the predictable latency mode (PLM) [6] has been added to the standard NVMe protocol to eliminate the problem by performing a closer collaboration between host and device. However, none of the industry-standard SSDs support this mechanism currently. Previous studies only validate PLM's effectiveness through simulation [28] or prototyping customized hardware to support PLM functionality [18, 21].

In this paper, we propose a SSD-based SIndex to maintain hundreds of billions of block-mapping entries in multiple SSDs with deterministic latency. SIndex performs comprehensive latency optimization across memory indexing, page caching and storage I/O. It

✉ Qiang Cao is the corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICPP '24, August 12–15, 2024, Gotland, Sweden

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1793-2/24/08
https://doi.org/10.1145/3673038.3673041

first designs a multi-level index to achieve high concurrency and stable query latency in memory. To alleviate the long-tail I/Os caused by SSD's internal tasks, SIndex proposes an inter-SSD scheduling that utilizes read/write separation and SSD state transition to mitigate the write-induced GC, and actively skips the buffer flushing based on performance profiling. SIndex further addresses the read latency spikes caused by read disturbance and parallel competition through dynamic read selection. The system-level SIndex schedules SSD's external I/O patterns on the host side, thus maintaining compatibility with commercial SSDs.

The main contributions of this paper are as follows.

- We experimentally analyze long-tail latency of KV stores built on commercial SSDs and reveal that the primary cause is the impact of SSD's internal tasks on I/O requests.
- We propose a system-level SIndex with deterministic latency to index billion-scale datasets. SIndex addresses the long-tail storage I/O by strategically separating and scheduling requests across multiple SSDs.
- We prototype and evaluate SIndex on a variety of commodity SSDs with real-world workloads. It consistently outperforms other approaches by up to 11.2 × in tail latency.

The rest of the paper is organized as follows. Section 2 presents the background for cloud block storage and their need of large-scale indexes. Section 3 analyzes the latency behaviors of KV stores and SSDs. Section 4 describes the design of SIndex. We evaluate SIndex in Section 5 and describe related works in Section 6.

2 BACKGROUND

2.1 Solid State Drive

Modern Solid State Drives (SSDs) with increasing performance and capacity have been gradually replacing Hard Disk Drives (HDDs) as the primary storage devices in many scenarios. The SSDs can add flash-channels/chips/dies/plane to augment intra-SSD parallelism [30], and the storage systems can further employ multiple SSDs to enhance I/O bandwidth and capacity [16]. Additionally, an SSD has to perform complex I/O handling logic such as page re-mapping [30], garbage collection [13, 34], wear-leveling and internal buffering/caching [14], resulting in unpredictable I/O latency.

2.2 Cloud Block Storage

Cloud block storage (CBS) systems provide virtual disks at block-accessibility for tenants with rigorous performance and availability requirements, which have been widely offered by cloud providers such as AWS [1], Alibaba [22], Google [9], etc. CBS with high request-level parallelism internally redirects all accesses of virtual blocks (*vblocks*) to physical blocks (*pblocks*) in the underlying SSD logfiles (SSDlogs) in an append-only manner [22, 23], thus fully leveraging SSD's bandwidth. The CBS system in China Mobile Group makes full use of adequate I/O parallelism by distributing user requests across clustered storage nodes (i.e., servers) and massive SSDlogs through a *vblock-pblock* mapping mechanism [25]. The virtual space available to users is initially segmented into multiple virtual planes (*vplanes*), each being 32GB by default and identified by a unique ID. Within each plane, CBS organizes multiple sequential *vblocks* into 256KB *vsectors*, which are distributed across SSDlogs through a hash-based algorithm.

Table 1: Evaluation Platform Specifications

Components	Configurations
Processor	Dual Socket Intel Xeon Gold 6133, 40 Cores, 56MB LLC
Memory	128GB 2133MHz DDR4
Operating System	Ubuntu 22.04.3 LTS with the Linux kernel version 6.2

Table 2: Characteristics of representative SSD products

Device Modules	Device Types	Write Thr. (kIOPS)	Read Thr. (kIOPS)	Write Lat. (μs)	Read Lat. (μs)	NVMe PLM	Media Types
P4610	NVMe	220	630	12	75	No	TLC
P4510	NVMe	70	460	16	75	No	TLC
970PRO	NVMe	500	370	13	65	No	MLC
860PRO	SATA	90	100	30	90	No	MLC

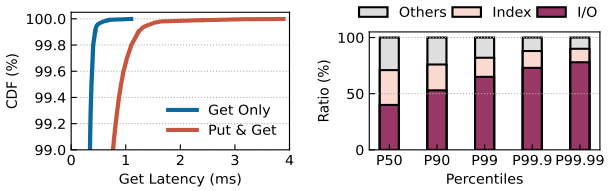
The *vblock-pblock* translation introduces a block mapping entry (**BM**) to track the storage location of each *vblock*. A BM constitutes a 48-Byte key-value pair (KV); the 8-Byte key contains *vplaneID* (4 Bytes), *vsectorID* (3 Bytes) and *vblockID* (1 Byte), and the 40-Byte value includes compressed crucial information for locating *pblock*. To enable efficient BM querying and updating in the critical I/O path, CBS maintains a BM manager utilizing classic memory index structures (e.g., Red-black Trees) to achieve low-latency performance. At runtime, CBS conducts batch insertions and deletions to the BM manager since it merely allocates storage space at the *vplane* granularity. Additionally, it primarily utilizes BM manager to search for the required *pblocks* with point or range queries.

2.3 Indexing for Billion-scale KV

Large-scale CBS system contains massive key-value pairs. For instance, a PB-level CBS needs about 300 billion BMs, occupying over 25 terabytes of memory. This typically surpasses the memory capacity of a single server, which is commonly less than 4TB due to a limited number of memory channels in a CPU socket (i.e., fewer than 8) and the restricted DRAM capacity (i.e., 256GB per DIMM). To address this memory limitation, CBS deploys an exclusive metadata cluster to provide enough memory and distributes all BMs across multiple servers [22]. However, this solution entails low-latency networks, effective load balancing among servers, and strict consistency guarantees, which are even more important and expensive than large memory capacity.

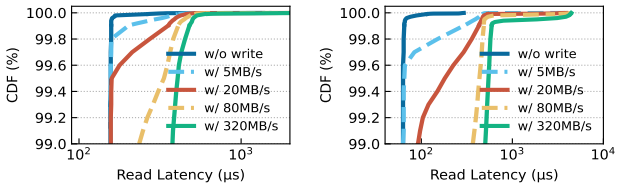
With growing performance and capacity of SSDs, it is anticipated that a single BM server equipped with multiple SSDs could support up to hundreds of billions of KVs. The log-structured merge tree (LSM-Tree) [27] is a memory-bound KV store to buffer incoming writes in memory and then flush them into storage in batch, which provides capacities orders of magnitudes larger than DRAM, thus striking a beneficial balance between performance and scalability. RocksDB [7] as a production LSM-tree is broadly used in cloud storage and high-performance computing [29, 32]. Besides, B-Tree based KV stores such as leanstore [20] and hash based approaches like vmcache [19] are designed to store KV pairs on SSDs.

Cloud storage services must ensure the Quality of Service (QoS) in latency. For example, commercial CBS promise an average request latency of under one millisecond [1]. Actually, the average read and write I/O latency of NVMe-based SSDs are prevalently lower than 100 and 50 microseconds, respectively. This presents an opportunity to employ cost-effective SSDs replacing DRAM to achieve the QoS objective. However, maintaining consistent low-latency indexing upon SSD-based KV stores is still unanswered.



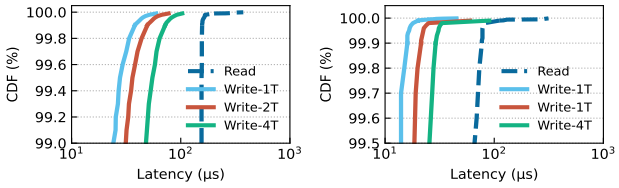
(a) CDF (b) Breakdown

Figure 2: Long-tail latency analysis of RocksDB



(a) P4610 (b) 970PRO

Figure 3: Impact of garbage collection on read latency



(a) P4610 (b) 970PRO

Figure 4: Effect of SSD internal buffer on write latency

3 ANALYSIS AND MOTIVATION

3.1 Understanding the Indexing Latency

We start with measuring the latency performance of LSM-tree based RocksDB as representative of SSD-based index systems. The platform configuration is detailed in Table 1. In these experiments, we build RocksDB on the P4610 SSD, whose I/O characteristics are listed in Table 2. We invoke a single user thread to execute Get-only and 1:1 Put/Get mixed workloads, respectively. Fig.2(a) shows the CDF of Get latency. In the Get-only scenario, latencies for all operations are maintained below 1.2 millisecond. However, in the mixed scenario, the tail-latency at the 99.9^{th} percentile (P99.9) and 99.99^{th} percentile (P99.99) increased by 2.3× and 14.1× than their normal state, respectively.

To investigate the sources of long-tail latency, we break down the execution time of Get operations under mixed workload into three parts: index query time (`Index`), storage I/O latency (`I/O`) and additional latencies caused by kernel-handling (`Others`). Fig.2(b) shows that the storage I/O for reading data from SSTfiles represents 51%-77% of the total execution time, dominating Get latency at the tails. Specifically, the I/O latency significantly increases when RocksDB initiates *flush* or *compaction* tasks that transfer MemTable to the on-disk tree or merge SSTfiles from higher to lower tree levels, respectively [2]. These operations induce considerable background I/Os and contention on SSDs, which in turn significantly impact the indexing performance, particularly for tail-end latencies.

Additionally, `Index` accounts for more than 28% of the Get operations under typical conditions (i.e., P50), and it constitutes over 15% of the latency beyond the P99 tails. This is mainly because RocksDB has to consecutively retrieve through Memtables in memory and index blocks in SSTfiles to locate the required key, leading to high query latency and even generating additional storage I/Os [5].

3.2 Long-tail I/Os on SSD

Modern SSDs exhibit variable I/O latency due to internal sophisticated I/O handling and maintenance tasks. We further perform a set of experiments to identify and understand the root cause of long-tail I/O latency on SSDs.

Garbage Collection. Flash media exhibits unique characteristics such as write-before-erase and asymmetric granularity between reads/writes and erasures [30, 33]. Consequently, SSDs direct incoming writes to new blocks rather than updating their original location. To release invalid data, the garbage collection (GC) is regularly executed to migrate valid pages from old blocks to a free block, subsequently releasing the entire old blocks. The GC process generally takes millisecond to complete and pauses incoming requests before its completion [13].

To evaluate the impact of GC on read I/O latency, we conduct 4KB-sized random read operations while performing writes at various throughput ranging from 5MB/s to 320MB/s in the background, which invokes GC tasks with varying intensities. We use the normal SSD read performance without writes as the baseline (labeled as `w/o write`). Fig.3 shows that the P99.9 tail read latency is increased by 8× than baseline under the minimal 5MB/s writes (0.2% of SSD's peak throughput). Besides, the intensities of GC gradually escalate as the write throughput increases, and over 18% of read requests endure high latencies exceeding 500μs under 320MB/s write-throughput. Previous works conditionally suspend or postpone GC [30, 33] to prioritize user accesses. Nevertheless, GC remains indispensable and affects user I/Os under continuous workload.

Buffer Flush. Modern commodity SSDs generally employ a built-in DRAM buffer to absorb writes due to a large latency gap between the buffer and backend flash media [30]. Fig.4 shows the latency CDF of random read and write workloads with varying threads on P4610 and 970PRO SSDs. Results indicate that the built-in buffer significantly enhances write performance. The average and P99 write latencies are less than 30μs and 80μs, which are substantially 3× and 6× compared to the read latency, respectively. In contrast, random read requests require direct access to the flash memory, exhibiting longer observed I/O latency.

However, SSDs should strategically flush data to the backend for reclaiming the buffer space, which triggers garbage collection and disturbs read requests during periods without incoming write operations. Fig.5 shows the impact of buffer flushing on read latency. We invoke two threads to concurrently issue 1MB and 4KB random writes and reads for 50 seconds respectively, and then stop writing while continuing to read. Results demonstrate that all SSDs still suffer from latency peaks even with the write workload suspended. Besides, the time taken to flush the buffer varies among different SSD models due to their own flushing strategies. For example, the observed flush durations are 7ms on P4610, 8ms on P4510, 10ms on 970PRO, and 4ms on 860PRO, respectively.

Read Disturbance. When reading a set of SSD pages continuously and repeatedly, the associated flash cells may experience an

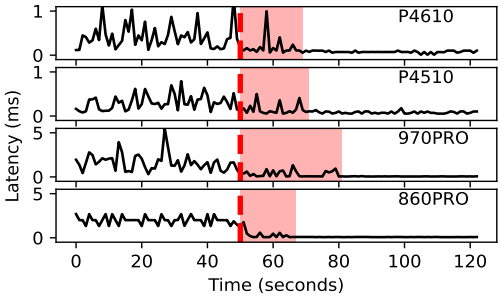
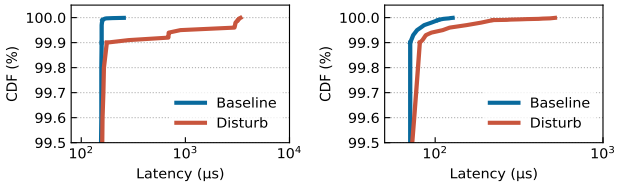


Figure 5: Impact of buffer flush on read latency



(a) P4610 (b) 970PRO

Figure 6: SSD Performance under read disturbance

elevated error rate as a result of charge drift [10]. To alleviate read disturbance, SSDs need to periodically transfer data from frequently accessed blocks to new blocks. In this experiment, we employ a single thread to repeatedly read a set of 4KB-sized blocks with fixed offsets on P4610 and 970PRO SSDs, respectively. We use the full-disk random read workload as the baseline.

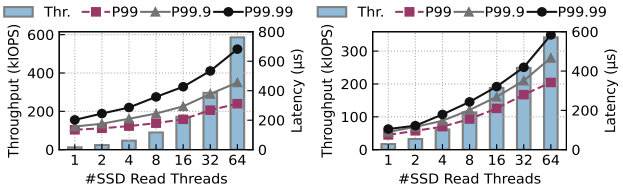
Fig.6 shows that the read disturbance causes the P99.9 and P99.99 tail latencies to increase by 5.5× and 13.2× respectively, with the peak latency exceeding 3.3ms. It demonstrates that long-tail I/O latency happens even in an optimal read-only workload. Additionally, we observe that the extent of read disturbance varies across SSDs due to the diverse flash media and data migration strategies employed by their manufacturers.

Request Parallelism. Current applications explicitly adopt multi-threaded I/O mode to exploit the high inherent parallelism of SSDs. In experiments, we invoke up to 64 threads to concurrently issue 4KB random reads on different SSDs. Fig.7 indicate that P4610 and 970PRO nearly reach the advertised peak read throughput (600 and 360 kIOPS) under 64 threads respectively, which achieve nearly 42× and 21× improvement over a single thread (14 and 17 kIOPS), respectively. However, the P99.9 and P99.99 tail-latencies with 16 threads are increased by up to 2.1× and 4.4× than a single thread, respectively. It suggests that highly parallelized I/O requests would compete for SSD resources such as request queue and internal flash channels, even when operating below the rated peak throughput.

In summary, servers utilizing SSDs to host large-scale KVs would experience long-tail latency issues, which primarily stem from the unpredictable I/O processing latency induced by SSD internal tasks and parallel I/O contention.

3.3 Efforts on Taming Tail-latency

The dramatic I/O performance variation has become a widely recognized challenge in the storage industry [11]. Many efforts have been made to address this issues, including device-layer modifications [17, 33], host-level optimizations [12, 17], programmable devices



(a) P4610 (b) 970PRO

Figure 7: SSD performance with different I/O parallelism

[3, 4] and host-device co-design approaches [6, 18, 21]. The device-layer solutions such as GC preemption and suspension [17, 33] require modifying the SSD firmware. Host-side methods such as latency predictions and I/O speculations [12, 17] could become less efficient under high-intensity or complex workloads, due to their inability to eliminate interference from internal SSD tasks.

Recently, the NVMe standard has introduced a Predictable Latency Mode (PLM) [6] to mitigate the tail-latency challenges. It suggests that the SSDs should identify deterministic windows (DTWIN) and non-deterministic windows (NDWIN) and report them to the host. During the DTWIN period, the device has to postpone any internal operations potentially causing latency fluctuations (e.g., garbage collection) to the NDWIN period. However, to the best of our knowledge, there exist no commercial SSDs that support PLM. This is primarily because PLM is an interface protocol rather than a design specification, which does not guide manufacturers to adjust firmware to support PLM features. Therefore, it is essential to propose a more adaptable and universally applicable approach for achieving stable I/O latency on commercial SSDs.

4 DESIGN

Given the identified causes of long-tail latency, we introduce SIndex, a novel SSD-based block mapping manager with deterministic latency in CBS system. To mitigate the long-tail SSD I/Os, SIndex proposes a write-staging buffer cache and an inter-SSD scheduling mechanism to actively elude the SSD with internal maintenance tasks. SIndex also presents a multi-level index structure to reduce indexing path with high concurrency.

4.1 Architecture

Fig.8 illustrates the architecture of SIndex. It is built upon multiple mirrored SSDs, each of which provisions a dedicated space for distributed WAL to enable quick recovery and persists SIndex's metadata in a MetaTable at the pre-defined location. SIndex integrates multiple BM entries into 4KB-aligned ipages and sequentially stores them on SSDs, and utilizes a multi-level index (§4.2) to provide high query performance by optimizing traversal complexity and mitigating lock contention under concurrent workloads.

The long-tail I/O latency of SSDs poses a key challenge in achieving stable indexing latency. SIndex first introduces a write-staging buffer cache (§4.3) with a two-stage sync mechanism to strategically buffer updated ipages to regulate the SSD write behavior, while caching hot ipages to minimize data fetches from underlying SSDs. Furthermore, instead of modifying SSD hardware, SIndex proposes an inter-SSD I/O scheduling that eliminates the impact of write-induced GC on reading ipages by strategically separating read and

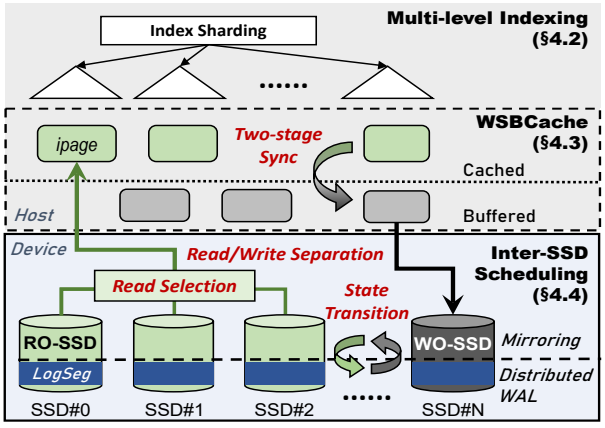


Figure 8: Architecture of SIndex

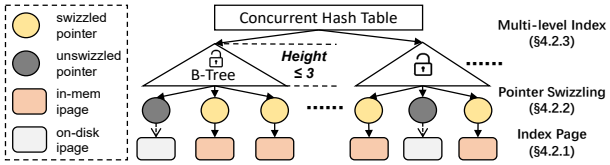


Figure 9: Two-level index structure

write requests to read-only SSDs (**RO-SSDs**) and a write-only SSD (**WO-SSD**) (§4.4.1). It further proposes an epoch-based state transition that periodically synchronizes updated ipages among all SSDs and reallocates SSDs' working states for wear-leveling, while actively bypassing the buffer flush during the transition process based on performance modeling. Moreover, SIndex introduces a dynamic read selection mechanism (§4.4.2) that opportunistically exploits the I/O capabilities of multiple RO-SSDs to address accidental latency spikes caused by read disturbance and parallel competition. Furthermore, a server can deploy multiple SIndex instances to manage more SSD devices, thereby achieving scalable storage capability.

4.2 Index Structure

Fig.9 illustrates the index structure of SIndex. To efficiently store hundreds of billions of BMs, SIndex consolidates multiple BM entries into index pages (ipages) and stores them to SSDs (§4.2.1), while allocating swizzle pointers in memory to provide fast page address translation (§4.2.2). Building on this, SIndex adopts a multi-level index (§4.2.3) to achieve short indexing path and high concurrency.

4.2.1 Index Page. SIndex integrates multiple BMs into the 4KB-aligned index page (**ipage**) that is congruent with the native storage granularity of SSD, optimizing disk utilization by preventing read-modify-write. As shown in Fig.10, *BM_array* stores all BM records belonging to the same *vsector* (i.e., 64) in order of the *vblockIDs*. This approach also exploits spatial locality to improve index performance for sequential and large-sized workloads in CBS.

In addition, each ipage contains self-descriptive information, such as *ipageID*, *vplaneID* and *vsectorID* to facilitate the reconstruction in cases when SIndex is corrupted. A *CRC* check is incorporated to verify data integrity. Furthermore, it records runtime information such as an *AccCount* that increments for each page access and a *Dirty* flag that is set after BM updates; a *SyncEP* field

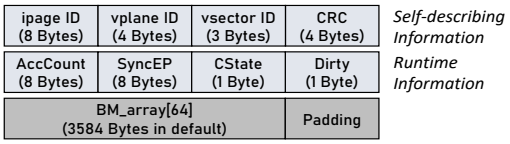


Figure 10: Structure of the ipages

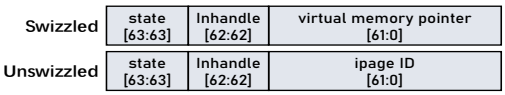


Figure 11: Structure of the swizzle pointer

records the last page sync time, and a *CState* flag represents the cached or buffered states of the ipage, which is critical for the page caching described in the next Section 4.3. In addition, SIndex will actively pad data to ensure 4KB alignment when serialized to SSD.

4.2.2 Page Address Translation. To look up or update BMs, SIndex has to fetch the SSD-stored ipages into memory and translate the *ipageID* to a virtual memory pointer. This address translation in traditional databases is typically done using a hash table that contains all pages currently cached in main memory, which leads to additional hash table lookup overhead [20].

SIndex utilizes the pointer swizzling [8] to overcome this problem by assigning a dedicated swizzle pointer (**swptr**) for each ipage. Fig.11 illustrates the structure of a *swptr*, which is represented as a 64-bit integer. The highest bit encodes a *state* field to indicate whether the associated ipage is in memory (i.e., swizzled) or on SSD (i.e., unswizzled). Additionally, it composes an *Inhandle* flag to signal whether the ipage is in I/O process, which can be updated atomically by executing the CAS (compare and swap) operation to prevent inconsistency among multiple user and/or worker threads.

4.2.3 Multi-level Indexing. SIndex designs a multi-level index (**MLI**) to efficiently query large-scale BMs. Specifically, the high-level is a concurrent Hash Table that uses the *vplaneID* field as the key to query BMs on different *vplanes*. This provides CBS system to batch BM insertions and deletions at the *vplane* granularity during virtual space reallocation by simply inserting/removing entries in the hash table, as mentioned in Section 2.2. In addition, the low-level utilizes B-Tree to enable efficient point and range queries. SIndex assigns a B-Tree instance to exclusively stores all the *vsectors* related to a *vplane* and uses a dedicated lock to minimize contention under concurrent accesses from various users. Besides, since each *vplanes* contains a fixed number of *vsectors* according to the *vblock*−*pblock* mapping mechanism (Section 2.2), SIndex precisely tunes the capacity of internal B-Tree nodes to ensure the tree height does not exceed 3, achieving consistently short indexing path.

4.3 Write-Staging Buffer Cache

Sindex presents a write-staging buffer cache (**WSBCache**) to strategically buffer updated ipages of the SSDs in read-only time window to separate read and write while caching frequently-accessed ipages. WSBCache employs multiple clock lists (i.e., 16 in default) organized as linked-list to collaboratively manage *swptrs* corresponding to cached ipages. Upon reading a new ipage from SSDs, WSBCache appends its *swptr* to the end of the shortest clock list to avoid imbalances in list length for reducing retrieval efficiency.

Algorithm 1 Cache Eviction

function Clock_Eviction()
1: **while** cache capacity exceeding $Cache_{size}$ **do**
2: $swptr \leftarrow$ clock_list.traverse()
3: atomic_sub($ipage.AccCount$)
4: **if** $ipage.AccCount == 0$ **then**
5: **if** $!ipage.Dirty$ **then**
6: free $ipage$ and unswizzle $swptr$
7: **else**
8: TwoStage_Sync($ipage$)

function TwoStage_Sync()
1: $ipage.CState \leftarrow buffered$
2: flush($ipage$, WO-SSD)
3: $ipage.Dirty \leftarrow$ false
4: **if** $ipage$ is synced **then**
5: **if** $!ipage.Dirty$ **then**
6: free $ipage$ and unswizzle $swptr$
7: **else**
8: $ipage.CState \leftarrow cached$

Fig.12 depicts the state transitions of an ipage (referred by circled numbers) during its life cycle. Initially, the *CState* field is set to the "cached" state once read from SSDs (❶), and updating contained BMs makes it become dirty (❷). When the cache capacity exceeds a threshold $Cache_{size}$, WSBCache executes concurrent cache eviction that assigns a dedicated thread for each clock list to perform clock-based eviction. As detailed in Algorithm 1 (referred by line number), all threads traverse their respective clock lists and decrement the *AccCount* field of each cached ipages (*lines 2-3*). Then, WSBCache releases clean ipages with a zeroed *AccCount* and unswizzles the corresponding $swptr$ (*lines 4-7*, ❸).

Furthermore, SIndex schedules the evicted dirty ipages to a dedicated WO-SSD without synchronizing to other RO-SSDs immediately, resulting in the inability to re-read these ipages with guaranteed low latency (detailed in the next Section 4.4.1). Therefore, different with traditional caching approaches that directly write dirty pages to disks upon eviction, WSBCache proposes a two-stage sync mechanism (TSS) that strategically buffers the evicted ipages in memory to provide stable latency of read-after-write operations, as shown in function $TwoStage_Sync()$ in Algorithm 1. In the first stage, WSBCache transfers ipages to the "buffered" state while flushing them to WO-SSD simultaneously, after which they are marked as clean (*lines 1-3*, ❹). After SIndex synchronizes buffered ipages to RO-SSDs, WSBCache enters the second stage that directly releases the clean ipages (*lines 5-6*, ❻), and the dirty ipages that have been re-updated during TSS (❺) will be transferred back to the "cached" state (*lines 7-8*, ❼), maintaining data consistency among SSDs.

The pre-allocate memory of WSBCache can be calculated by $Cache_{size} = Max_{Band} * T_{epoch}$, where Max_{Band} represents the bandwidth of WO-SSD and T_{epoch} is the sync interval (5 seconds in default). In most cases, the space usage of WSBCache is less than 10GB for each SIndex instance, which can be accommodated by servers with hundreds of GBs memory.

4.4 Inter-SSD I/O Scheduling

As analyzed in Section 3.2, the internal tasks in SSDs induce significant tail I/O latency. Fortunately, we get an insight that most of them could be mitigated or even avoided through merely regulating SSD's external I/O patterns on the host side. Therefore, SIndex incorporates a latency deterministic inter-SSD I/O scheduling (**LDIS**)

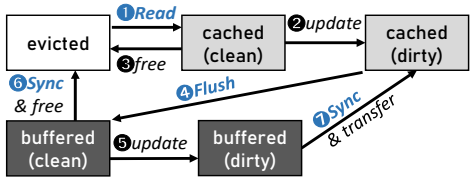


Figure 12: The state transitions of ipages

that mitigates the write-induced GC by request separation and actively skips the buffer flushing based on performance profiling. LDIS further improves the read-only latency by addressing the read disturbance and parallel competition through a dynamic read selection, while maintaining compatibility with commercial SSDs.

4.4.1 Epoch-based State Transition. Through prior analysis, we find that the intensive GC task is only triggered by write requests instead of read. Therefore, SIndex employs a simple and efficient read-write separation method that first partitions multiple SSDs into several RO-SSDs and a single WO-SSD, and then redirects read requests to the RO-SSDs and sends all cache flushes (i.e., write requests) to the WO-SSD. Different from approaches that postpone and/or preempt GC tasks [17, 33], SIndex fundamentally prevents triggering GC on RO-SSDs by scheduling external I/O patterns.

Nonetheless, SIndex should periodically switch the states of SSDs for wear leveling. It introduces an epoch-based state transition that maintains SSDs in either a write-only or read-only mode throughout the epoch duration, then switches SSD states and synchronizes data across all SSDs following an epoch timeout. However, transitioning an RO-SSD to WO-SSD would compromise subsequent read latency due to the impact of internal buffer flushing. To address this, SIndex employs an *Evading Flush Delay (EGD)* during transition to proactively bypass buffer flushing through idling an SSD for a certain time until latency becomes stable. Since the buffer configuration varies across different SSD products, we measure the buffer flush duration of SSDs and set it as the *EGD* time through an experimental profiling method as mentioned in Section 3.2.

Furthermore, SIndex requires to periodically synchronize updated data among all the RO-SSDs, thereby triggering GC as well. To maintain consistent read latency, SIndex meticulously regulates SSDs' states to ensure there is always at least one RO-SSD available at any given time. Fig.13 (referred by circled numbers) and Algorithm 2 (referred by line numbers) describe the state transition using an example with three SSDs, where SSD#2 is WO-SSD during the last epoch. After an epoch timeout, SIndex sets the current WO-SSD to the waiting state and randomly selects a RO-SSD to take over as the new WO-SSD, which is ready to handle incoming write operations (*lines 1-2, Time* ①). Next, SIndex idles the waiting SSD for *EGD* to skip the buffer flush, then transfers it to RO-SSD for exclusively handling user read requests during the upcoming sync process (*lines 3-5, Time* ②). Subsequently, SIndex propagates buffered ipages from WSBCache to old RO-SSDs and instructs them to wait for EGD (*lines 6-8, Time* ③). Finally, SIndex reactivates the RO-SSDs and starts a new epoch (*lines 9-10, Time* ④).

4.4.2 Dynamic Read Selection. When reading from RO-SSDs, SIndex still has to address latency fluctuations caused by parallel I/O contention. In contrast to traditional RAID-1 that merely serves reads from the primary device, SIndex employs a request balancing

Algorithm 2 Epoch-based State Transition

```
1: wait_array.push(WO-SSD)
2: WO-SSD ← RO_array.pop(rand() % RO_array.size())
3: sleep(EGD)
4: sync_array.push(RO_array)
5: clear RO_array, then swap RO_array and wait_array
6: SIndex.sync(sync_array and WO-SSD)
7: wait_array.push(sync_array)
8: sleep(EGD)
9: RO_array.push(wait_array)
10: atomic_add(Epoch_number)
```

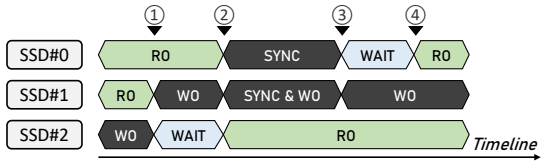


Figure 13: Workflow of state transition

method that dispatches requests to an RO-SSD with the shortest submission queue (SQ), thereby uniformly distributing intensive reads across multiple devices. Compared to the static request distribution approaches utilizing hash function, this simple and effective dynamic strategy can prevent directing requests to the slow SSD with a long queue length.

Furthermore, SIndex utilizes I/O speculation [12] including cloning and hedging to address the long-tail I/O induced by read disturbance. As shown in Fig.14, the request cloning submits I/O requests asynchronously to multiple SSDs (ⓐ) and reaps the first response from the completion queue (ⓑ), effectively avoiding excessive long-latency for a specific request. However, this could double the I/O traffic and inherently conflict with the request balancing method that aims at reducing I/O burden on SSDs. In contrast, the hedging method only re-sends requests to another SSD upon the first one's timeout (Ⓐ–Ⓒ), efficiently improving SSD utilization with the cost of doubled tail latency compared to the average.

To take advantage of both request balancing and I/O speculation under various workloads, SIndex proposes a Dynamic Read Selection (DRS) that selects the optimal scheduling method based on the read traffic of RO-SSDs. Specifically, SIndex monitors the throughput on reading ipages ($Read_{th}$) and utilizes request cloning under low-intensive workloads, where $Read_{th}$ is lower than an experimentally determined threshold T_{low} that is set to the single-threaded random read throughput on a single RO-SSD. Moreover, at moderate read intensities ($Read_{th} > T_{low}$), DRS enables request cloning and balancing simultaneously to mitigate parallel competition. When $Read_{th}$ exceeds the total random throughput of all RO-SSDs (T_{high}), DRS substitutes request cloning for hedging to significantly reduce I/O traffic on individual SSDs. In addition, DRS can be stopped during the epoch sync as only one RO-SSD is activated at this time.

4.5 Crash Consistency

SIndex periodically synchronizes mirrored data across SSDs during state transition and restores data from other SSD copies when any SSD fails. However, an incomplete sync process may lead to data inconsistency between SSDs. To this end, SIndex uses Write-Ahead Logging (WAL) to ensure recoverability by recording all BM updates into the WAL before overwriting their related ipage. As shown in

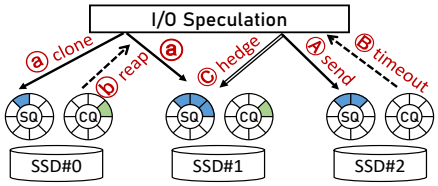


Figure 14: Workflow of the I/O speculation in SIndex

previous Fig.4, the SSD built-in buffer will absorb the WAL requests and provide stable write latency.

However, the log-induced writes would trigger unexpected garbage collections on SSDs. SIndex employs a *distributed WAL* that deploys an individual WAL Segments (LogSeg) on each member SSDs and exclusively writes to the LogSeg on WO-SSD at runtime. In contrast to the traditional logging approach that typically stores all records in a single file, the distributed WAL scatters records across all LogSegs due to the state transition at runtime. SIndex captures the current *Epoch_number* in each logging page and leverages these identifiers to reorder the LogSegs and replay recorded BM updates on system recovery. By default, SIndex reserves 20GB for each LogSeg, representing 0.5% of a 4TB SSD. Moreover, SIndex utilizes the mature multi-replica mechanism that stores WAL replica on other servers to prevent the single point of server failure in production system.

SIndex utilizes the MetaTable to rebuild the in-memory multi-level index after a system crash. The MetaTable records the *vplaneID* and the storage offset of ipages for each *vplanes*. SIndex retrieves the persistent MetaTable in SSD to identify the range of *ipageIDs* for *vplanes* and initialize *swptrs* in memory. Then, it recovers the index structure by reconstructing the b-tree instances and inserting their root pointers to the high-level hash table.

5 EVALUATION

5.1 Evaluation Setup

Platform. We validate the performance of SIndex using four types of commodity SSD products as described in Table 2 and run experiments on a storage server with detailed settings listed in Table 1. In our experiments, we bind all the I/O threads and worker threads to different CPU cores to avoid context switching overhead.

System setup. We evaluate `SIndex` with the popular `RocksDB` and two state-of-the-art SSD-based indexes, `leanstore` [20] and `vmcache` [19]. We deploy all systems based on three SSDs. SIndex organizes SSDs in the mirrored layout and other systems use RAID-0 to achieve the best performance on our platform.

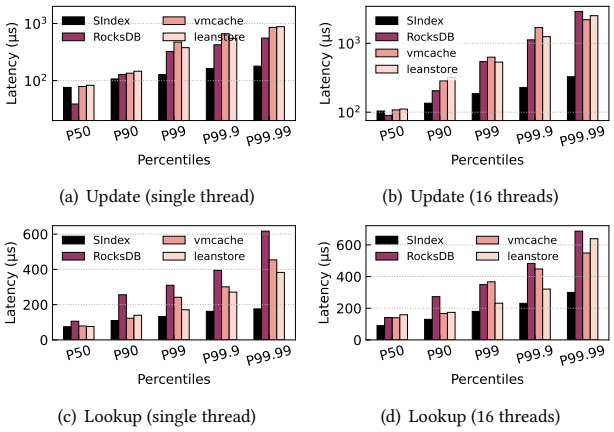
Workloads. We issue I/O requests with sequential or random access patterns as micro-benchmarks, and further select six representative block traces summarized in Table 3 as trace-driven macro-benchmarks. We implement a program that converts block requests on the virtual space into BM lookups or updates on the index systems. In evaluation, we initialize systems by inserting BMs for 12k *vplanes* (i.e., about 100B BMs and 6TB storage), then run each experiment three times and take the average as the results.

5.2 Micro-benchmark

Latency. Fig.15 shows the latency performance under lookup and update workloads, respectively. Note that the y-axis is set to log-scale in Fig.15(a) and Fig.15(b). SIndex reduces the P99 and P99.9

Table 3: Characteristics of block I/O traces

Trace	Block Ops (millions)	Read/Write (%)	Avg. Read Size(KB)	Avg. Write Size(KB)	Random (%)
Ali-A	353	83 / 17	153.7	25.6	90.9
Ali-B	160	90 / 10	41.6	19.0	90.3
Tenc-A	230	48 / 52	24.6	36.5	96.1
Tenc-B	265	28 / 72	28.8	38.0	95.5
prxy1	93	65 / 35	12.3	12.9	84.0
prn1	56	75 / 25	22.5	11.7	77.5



(a) Update (single thread) (b) Update (16 threads)

(c) Lookup (single thread) (d) Lookup (16 threads)

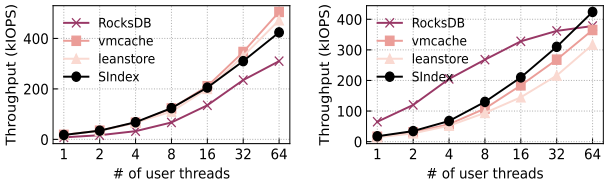
Figure 15: Latencies on different indexes (lower is better)

tail latency for update operations by 3.5× to 11.2× compared to other approaches, respectively. This improvement is primarily due to the request separation in SIndex that reads data from RO-SSDs and flushes to the dedicated WO-SSD, avoiding performance degradation caused by write-induced GC. Furthermore, under single-threaded lookups that involve no SSD writes, the P99 and P99.9 tail latencies of SIndex are 1.5× and 2.2× lower than other methods respectively, which is attributed to the fine-grained I/O speculation for mitigating latency spikes caused by read disturbance. With 16 threads, SIndex further utilizes the request balancing to exploit the inter-SSD parallelism to reduce contention between intensively concurrent I/O operations.

Additionally, RocksDB utilizes an out-of-place update method that first inserts BM entries into the in-memory MemTable and then flushes them to the disk by background compaction [2], thereby achieving optimal update performance. However, the update latency above P99.9 increases sharply with 16 threads due to WAL and MemTable bottlenecks under concurrent workloads [24].

Throughput. Fig.16 shows that the lookup throughput is basically identical between SIndex and other approaches under lower user threads. However, it decreases by up to 17% and 11% compared to vmcache and leanstore under 64 threads, respectively. This is mainly because SIndex assigns a dedicated WO-SSD for separating write operations, thus reducing the number of RO-SSDs available to handle concurrent workloads. Additionally, SIndex employs request cloning and/or hedging to optimize tail latency, at the expense of a slight increase in read traffic.

In the update scenario, SIndex outperforms vmcache and leanstore by 12% and 28% respectively due to mitigating the GC-induced read penalties. SIndex demonstrates minimal performance difference on updates compared to lookups, while other approaches experience a 35% decline in update throughput than that of lookup. Moreover,



(a) Lookup (b) Update

Figure 16: Throughput on different indexes

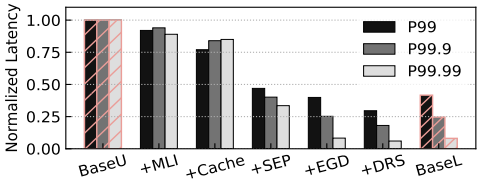


Figure 17: Effect of the main features in SIndex on latency

RocksDB achieves higher write throughput through the out-of-place update method, but it compromises the lookup performance by an average of 30% compared to others.

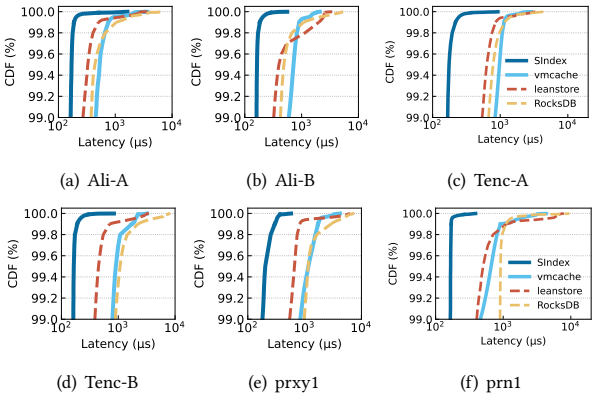
Performance Breakdown. To better understand the impact of SIndex's design on tail latency, we conduct a traditional SSD-based index system that replaces the multi-level index with a single b-tree and uses RAID-0 to manage underlying SSDs. In experiments, we invoke 16 threads to execute concurrent update (labeled as `BaseU`) and lookup (labeled as `BaseL`) workloads, respectively.

Fig.17 shows the performance of SIndex when its crucial features are enabled step by step. The multi-level index (`MLI`) reduces P99.9 and P99.99 latancies by 6% and 11% respectively by reducing long-path queries and mitigating concurrent lockdown. In addition, the pointer swizzling eliminates the overhead of retrieving mapping tables at each ipage access. The write-stage buffer caching (`Cache`) reduces the P99 and P99.9 latency by up to 16% and 8%. This is due to the cache's ability to absorb hot BM accesses for alleviating the I/O burden on SSDs. WSBCache also ensures stable latency for read-after-write by buffering evicted ipages in memory.

The inter-SSD I/O scheduling is the largest factors that reduces up to 48% tail latency caused by long storage I/O. First, the request separation (`SEP`) mitigates the impact of write-induced GC on reading ipages, while the evading GC delay (`EGD`) greatly decreases the latency above P99.9 by skipping the buffer flush during state transitions. With the previously mentioned features activated, SIndex achieves tail latency similar to `BaseL` even under update workloads. Further, the dynamic read selection (`DRS`) utilizes I/O speculation and request balancing to effectively minimize latency spikes stemming from parallel competition and read disturbance, leading to a 17% latency reduction compared to `BaseL`.

5.3 Macro-benchmark

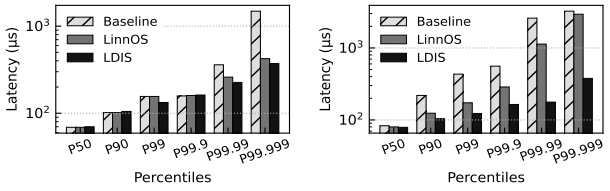
We use six representative cloud-based block traces from AliCloud [23], Tencent [35] and MSRC [26] to evaluate SIndex's performance. Table 3 summarizes the characteristics of these workloads, most of which are read-dominated (Ali-A and Ali-B) and read/write mixed (Tenc-A, Tenc-B, prn1 and prxy1). In experiments, we invoke 16 threads to replay these traces and transform them into BM lookup and update operations, mimicking high-intensity workloads.



(a) Ali-A (b) Ali-B (c) Tenc-A

(d) Tenc-B (e) prxy1 (f) prn1

Figure 18: Latency CDF on trace-driven workloads



(a) Read Only (b) Read/Write Mixed

Figure 19: Latency of LDIS and LinnOS

Fig.18 shows that SIndex significantly outperforms other approaches across all the workloads. For the mixed workloads, SIndex performs at least 3.5×, 8.4× and 11.1× lower P99, P99.9, and P99.99 latency reduction. The primary source of these long latency is the GC-induced writes on SSDs, which could be efficiently mitigated by the read-write separation in SIndex. For the read-dominated workloads, the tail latencies of SIndex are still 2.2× to 4.8× lower than those of other systems. This is because previous experiments (§3.2) have shown that even a small number of writes could trigger garbage collection and greatly affect read performance.

5.4 Sensitive Study

Comparison with block-level approaches. We evaluate the effectiveness of SIndex and LinnOS [12] on reducing I/O latency. LinnOS predicts the slowdown of each I/O by exploiting a neural network and performs hedging on failed predictions. To make a fair comparison, we extract the LDIS functionalities from SIndex and integrate them into a block-level I/O scheduling system. We use three P4610 SSDs with mirrored layouts and conduct a single thread to execute 4KB random reads or mixed read-write workloads. We use the traditional RAID-1 as a baseline.

Fig.19 shows that the difference between LDIS and LinnOS is less than 9% under read-only conditions. However, the P99.9 and P99.99 latency of LinnOS is 1.9× and 8.8× higher than LDIS under mixed workloads, respectively. This is because LinnOS and RAID-1 write data to all SSDs to maintain consistency, leading to a high probability of simultaneously triggering GC in all SSDs. In such cases, latency prediction or I/O speculation alone is rendered ineffective. In contrast, LDIS utilizes request separation to prevent initiating GC in RO-SSDs and employs per-I/O cloning or hedging to mitigate occasional high latency caused by read disturbance.

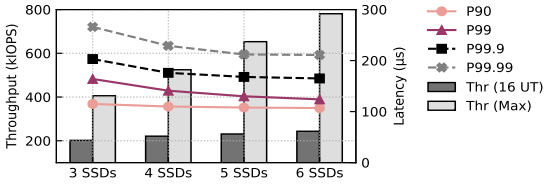


Figure 20: SIndex on different number of SSDs

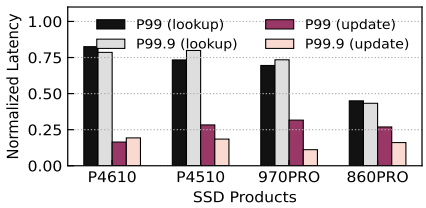


Figure 21: Latency of SIndex on various SSD products

Experiment with more SSDs. Next, we evaluate the performance scalability of SIndex built on different numbers of SSDs, ranging from three to six P4610 SSDs (labeled as `3 SSDs` to `6 SSDs`). In this experiment, we invoke 16 user threads executing random lookup operations for testing the tail-latency and use 64 threads to measure the maximum throughput. Fig.20 shows that P99.9 and P99.99 latencies with *6 SSDs* decrease by 18% and 21% compared to those with *3 SSDs* at 16 threads, respectively. This is because the request balancing mechanism in LDIS efficiently leverages the inherent parallelism among multiple RO-SSDs. Besides, the peak throughput of SIndex increases by up to 1.7× with *6 SSDs* at 64 threads, indicating that SIndex can fully exploit the high throughput of multiple SSDs without decreasing latency performance.

Experiment with various SSD products. We evaluate the sensitivity of SIndex on four types of SSD products as listed in Table 2, and invoke 16 threads to issue random lookup and update operations respectively. We measure the P99 and P99.9 latencies and normalize them to SIndex built on RAID-1 (i.e., disable LDIS). Fig.21 shows that SIndex consistently reduces the tail latencies by 28% and 76% in average under lookup and update workloads respectively, because it significantly mitigates the impact of SSD internal tasks on user requests. Moreover, SIndex built on 860PROs achieves more latency reduction compared to other SSDs. The SATA-based 860PROs have a much lower peak throughput than other NVMe-based SSDs, as detailed in Table 2. Therefore, the request balancing mechanism could fully exploit the inter-SSD parallelism to reduce high I/O latency.

6 RELATED WORKS

Device-level Modification. Previous studies address the SSD latency issues caused by internal tasks, including device-level modifications [17, 33], interface optimizations [21] and programmable devices [3, 4]. PreemptGC [33] reduces GC-induced overhead by allowing user requests to preempt GC tasks. However, it cannot indefinitely postpone the garbage collection and will revert to blocking behavior under intensive workloads. Besides, they require extensive firmware re-architecting that could not be quickly deployed on commercial SSDs. Compared with that, SIndex considers SSDs

as black boxes and performs I/O scheduling at host-side to alleviate GC interference, making it highly portable and non-intrusive.

Moreover, VigilKV [18] enables NVMe PLM on a prototype SSD and schedules RocksDB's internal operations with the awareness of PLM features. IODA [21] extents the standard PLM interface to provide fine-grained latency predictability. However, they still face challenges in real-world application due to the lack of PLM-supported SSD products. SIndex mitigates the long-tail latency from storage I/Os within the host-side, avoiding the need for customized hardware and interface modifications.

Host-side Latency Optimization. Many efforts are made to exploit SSD's characteristics for KV stores [2, 24] and RAID systems [11, 16, 34]. GC-steering [34] redirects requests targeting the SSD undergoing GC to a dedicated spare SSD in RAID system. SWAN [16] alleviates the impact of GC in the all-flash-array system by using parts of the SSDs to serve writes and scheduling others to perform GC in the background. Compared to them, SIndex leverages the request separation method to mitigate GC interference, further utilizing I/O speculation and request balancing to alleviate read disturbance and parallel competition on read requests.

SSDCheck [15] constructs a performance model for predicting SSD I/O latency. LinnOS [12] proposes a neural network to predict the slowdown of each read request. However, the latency prediction would become less efficient under write-intensive workloads due to the fact that all SSDs are engaged in garbage collection simultaneously. SIndex utilizes request separation to prevent triggering GC among all the SSDs, and then strategically utilizes I/O speculation on RO-SSDs to mitigate the occasional read disturbances.

7 CONCLUSION

This paper introduces a SSD-based KV store SIndex with latency deterministic feature for CBS systems. To address long-tail SSD I/Os, SIndex employs request separation to alleviate write-induced GC, and strategically uses I/O speculation along with request balancing to mitigate the impact of read disturbances and parallel competition. In addition, SIndex utilizes the multi-level index to prevent long-path traversal and concurrent lock contention. Through trace-driven evaluations, SIndex is shown to outperform RocksDB and other approaches by up to 11.2× in tail latency.

ACKNOWLEDGMENTS

We would like to thank the anonymous reviewers for their valuable suggestion. This work was supported by NSFC No. 62172175 and Key Research and Development Project of Hubei No. 2022BAA042.

REFERENCES

[1] Amazon. 2023. Amazon EBS features. https://aws.amazon.com/ebs/features.
[2] Oana Balmau, Florin Dinu, Willy Zwaenepoel, et al. 2019. SILK: Preventing Latency Spikes in Log-Structured Merge Key-Value Stores. In *USENIX ATC 2019*. USENIX Association, 753–766.
[3] Matias Bjørling, Abutalib Aghayev, Hans Holmberg, et al. 2021. ZNS: Avoiding the Block Interface Tax for Flash-based SSDs. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, 689–703.
[4] Matias Bjørling, Javier Gonzalez, and Philippe Bonnet. 2017. LightNVM: The Linux Open-Channel SSD Subsystem. In *FAST 17*. USENIX, 359–374.
[5] Niv Dayan and Stratos Idreos. 2018. Dostoevsky: Better Space-Time Trade-Offs for LSM-Tree Based Key-Value Stores via Adaptive Removal of Superfluous Merging. In *SIGMOD Conference 2018*. ACM, 505–520.
[6] NVM Express. 2018. The NVMe 1.4 Specifications. https://nvmexpress.org/.
[7] Facebook. 2022. RocksDB. https://rocksdb.org/.
[8] Hector Garcia-Molina, Jeffrey D. Ullman, and Jennifer Widom. 2009. *Database systems - the complete book (2. ed.)*. Pearson Education.
[9] Google. 2024. Persistent Disk. https://cloud.google.com/persistent-disk.
[10] Keonsoo Ha, Jaeyong Jeong, and Jihong Kim. 2016. An Integrated Approach for Managing Read Disturbs in High-Density NAND Flash Memory. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 35, 7 (2016), 1079–1091.
[11] Mingzhe Hao, Gokul Soundararajan, Deepak R. Kenchammana-Hosekote, et al. 2016. The Tail at Store: A Revelation from Millions of Hours of Disk and SSD Deployments. In *FAST 2016*. USENIX Association, 263–276.
[12] Mingzhe Hao, Levent Toksoz, Nanqinqin Li, et al. 2020. LinnOS: Predictability on Unpredictable Flash Storage with a Light Neural Network. In *OSDI 2020*. USENIX Association, 173–190.
[13] Myoungsoo Jung, Wonil Choi, Miryeong Kwon, et al. 2020. Design of a Host Interface Logic for GC-Free SSDs. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 39, 8 (2020), 1674–1687.
[14] Woon-Hak Kang, Sang-Won Lee, Bongki Moon, et al. 2014. Durable write cache in flash memory SSD for relational and NoSQL databases. In *SIGMOD 2014*. ACM, 529–540.
[15] Joonsung Kim, Kanghyun Choi, Wonsik Lee, and Jangwoo Kim. 2021. Performance Modeling and Practical Use Cases for Black-Box SSDs. *ACM Trans. Storage* 17, 2 (2021), 14:1–14:38.
[16] Jaeho Kim, Kwanghyun Lim, Youngdon Jung, et al. 2019. Alleviating Garbage Collection Interference Through Spatial Separation in All Flash Arrays. In *USENIX ATC 2019*. USENIX Association, 799–812.
[17] Shine Kim, Jonghyun Bae, Hakbeom Jang, et al. 2019. Practical Erase Suspension for Modern Low-latency SSDs. In *USENIX ATC 2019*. USENIX Association, 813–820.
[18] Miryeong Kwon, Seungjun Lee, Hyunkyu Choi, et al. 2022. Vigil-KV: Hardware-Software Co-Design to Integrate Strong Latency Determinism into Log-Structured Merge Key-Value Stores. In *ATC 2022*. USENIX Association, 755–772.
[19] Viktor Leis, Adnan Alhomssi, Tobias Ziegler, et al. 2023. Virtual-Memory Assisted Buffer Management. *Proc. ACM Manag. Data* 1, 1 (2023), 7:1–7:25.
[20] Viktor Leis, Michael Haubenschild, Alfons Kemper, and Thomas Neumann. 2018. LeanStore: In-Memory Data Management beyond Main Memory. In *ICDE 2018*. IEEE Computer Society, 185–196.
[21] Huaicheng Li, Martin L. Putra, Ronald Shi, et al. 2021. IODA: A Host/Device Co-Design for Strong Predictability Contract on Modern Flash Storage. In *SOSP '21*. ACM, 263–279.
[22] Qiang Li, Qiao Xiang, Yuxin Wang, and othersothers. 2023. More Than Capacity: Performance-oriented Evolution of Pangu in Alibaba. In *FAST 2023*. USENIX Association, 331–346.
[23] Shuyang Liu, Shucheng Wang, Qiang Cao, et al. 2019. Analysis of and Optimization for Write-dominated Hybrid Storage Nodes in Cloud. In *SoCC 2019*. ACM, 403–415.
[24] Ziyi Lu, Qiang Cao, Hong Jiang, et al. 2022. p2KVS: a portable 2-dimensional parallelizing framework to improve scalability of key-value stores on SSDs. In *EuroSys '22*. ACM, 575–591.
[25] James W. Mickens, Edmund B. Nightingale, Jeremy Elson, et al. 2014. Blizzard: Fast, Cloud-scale Block Storage for Cloud-oblivious Applications. In *NSDI 2014*. USENIX Association, 257–273.
[26] Dushyanth Narayanan, Austin Donnelly, and Antony I. T. Rowstron. 2008. Write Off-Loading: Practical Power Management for Enterprise Storage. In *FAST 2008*. USENIX, 253–267.
[27] Patrick E. O'Neil, Edward Cheng, Dieter Gawlick, et al. 1996. The Log-Structured Merge-Tree (LSM-Tree). *Acta Informatica* 33, 4 (1996), 351–385.
[28] Tanaya Roy, Jit Gupta, et al. 2021. PLMC: A Predictable Tail Latency Mode Coordinator for Shared NVMe SSD with Multiple Hosts. In *NAS 2021*. IEEE, 1–6.
[29] Dipti Shankar, Xiaoyi Lu, and Dhabaleswar K. Panda. 2017. High-Performance and Resilient Key-Value Store with Online Erasure Coding for Big Data Workloads. In *ICDCS 2017*. IEEE Computer Society, 527–537.
[30] Arash Tavakkol, Mohammad Sadrosadati, Saugata Ghose, et al. 2018. FLIN: Enabling Fairness and Enhancing Performance in Modern NVMe Solid State Drives. In *ISCA 2018*. IEEE Computer Society, 397–410.
[31] Rui Wang, Yongkun Li, Hong Xie, et al. 2020. GraphWalker: An I/O-Efficient and Resource-Friendly Graph Analytic System for Fast and Scalable Random Walks. In *USENIX ATC 2020*. USENIX Association, 559–571.
[32] Teng Wang, Adam Moody, Yue Zhu, et al. 2017. MetaKV: A Key-Value Store for Metadata Management of Distributed Burst Buffers. In *IPDPS 2017*. IEEE Computer Society, 1174–1183.
[33] Guanying Wu and Xubin He. 2012. Reducing SSD read latency via NAND flash program and erase suspension. In *FAST 2012*. USENIX Association, 10.
[34] Suzhen Wu, Weidong Zhu, Yingxin Han, et al. 2020. GC-steering: GC-aware request steering and parallel reconstruction optimizations for SSD-based RAIDs. *IEEE TCAD* 39, 12 (2020), 4587–4600.
[35] Yu Zhang, Ping Huang, Ke Zhou, et al. 2020. OSCA: An Online-Model Based Cache Allocation Scheme in Cloud Block Storage Systems. In *USENIX ATC 2020*. USENIX Association, 785–798.